engineering proceedings

MDPI

Proceeding Paper

REST API Fuzzing Using API Dependencies and Large Language Models †

Chien-Hung Liu [1], Shu-Ling Chen [2,*] and Kuang-Yao Li [1]

[1] Department of Computer Science and Information Engineering, National Taipei University of Technology, Taipei 106344, Taiwan; cliu@ntut.edu.tw (C.-H.L.); t111598038@ntut.org.tw (K.-Y.L.)
[2] Department of Industrial Management and Information, Southern Taiwan University of Science and Technology, Tainan City 710301, Taiwan
* Correspondence: slchen@stust.edu.tw
† Presented at 8th International Conference on Knowledge Innovation and Invention 2025 (ICKII 2025), Fukuoka, Japan, 22–24 August 2025.

Abstract

With the widespread adoption of cloud services, ensuring the quality and security of the representational state transfer application programming interface (REST API) has become critical. Among various REST API testing techniques, fuzz testing stands out as a promising approach due to its ability to automatically generate large volumes of random or malformed inputs. To improve test coverage through fuzzing, we developed an enhanced method for generating API sequences and parameter values, building upon the widely used open-source tool RESTler. The approach extends RESTler by incorporating resource-level dependencies between APIs in addition to the existing producer–consumer relationships, enabling the construction of more valid API sequences. It also leverages a large language model to automatically generate parameter values. To further ensure input validity, a feedback loop is introduced to refine invalid inputs using error messages from API responses. Experimental results show that, compared to RESTler, the proposed method increases API coverage and detects more faults on average, demonstrating its effectiveness.

Keywords: REST API testing; fuzz testing; API dependency; REST API fuzzing; automated software testing

check for updates

Academic Editors: Teen-Hang Meen, Chun-Yen Chang and Cheng-Fu Yang

Published: 3 February 2026

Citation: Liu, C.-H.; Chen, S.-L.; Li, K.-Y. REST API Fuzzing Using API Dependencies and Large Language Models. *Eng. Proc.* **2025**, *120*, 42. https://doi.org/10.3390/engproc2025120042

Copyright: © 2026 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

Recently, Cloud services have matured significantly, with major companies including Amazon [1], Google [2], and Microsoft [3], which are actively developing and offering application programming interface (API)-based services. Most APIs are documented using the OpenAPI specification v3.1 (formerly known as Swagger) [4], and many follow the REpresentational State Transfer (REST) [5] architecture, which is especially common in cloud environments. These specifications include detailed information such as endpoints, requests, parameters, and responses—making it easier for both developers and machines to understand and interact with the services.

As REST APIs become increasingly popular and widely adopted across industries, concerns about their security and reliability are also growing. One of the most effective techniques for testing REST APIs is fuzz testing—a dynamic testing method that sends unexpected or random inputs to APIs to uncover vulnerabilities and potential bugs. Unlike traditional functional testing, fuzz testing can expose edge cases and hidden issues that

might otherwise go undetected. Recently, it has emerged as a powerful tool for identifying security flaws and ensuring the robustness of API implementations.

However, most existing API testing tools heavily rely on randomly generated input values, which makes it difficult to effectively test APIs that require specific formats or constrained input parameters. Furthermore, since many APIs are stateful—where later requests depend on resources created by earlier ones—it is essential to test them in sequence. Random input generation, without incorporating feedback from previous responses, often fails to support such dependencies, making it ineffective for exercising meaningful API call sequences.

Among stateful fuzz testing tools for REST APIs, one of the most widely recognized open-source solutions is Microsoft's RESTler [6]. RESTler explores deeper API states by inferring and constructing API sequences based on producer–consumer relationships. However, RESTler can occasionally generate invalid or redundant sequences—for instance, by repeatedly chaining the same API or combining unrelated APIs within a single sequence. These issues reduce testing efficiency and lead to the execution of semantically incorrect sequences. Additionally, RESTler uses a dictionary-based approach to generate parameter inputs. This random strategy often fails to produce valid inputs that satisfy parameter constraints, limiting its ability to reach deeper execution paths and resulting in lower test coverage.

To address the issues, we extended RESTler by integrating resource dependency analysis into API sequence construction and leveraging large language models (LLMs) for generating parameter inputs. Specifically, for sequence generation, the approach considers the producer–consumer relationships and resource-level dependencies to construct semantically valid and meaningful API sequences. For parameter generation, LLM is guided using carefully crafted prompts. Techniques such as name completion and parameter dependency analysis are applied to help LLM generate input values that precisely mitigate parameter constraints, enabling more reliable API execution. Additionally, when an API call fails during testing, the resulting error response is incorporated into a feedback prompt to guide LLM in refining the input values. This iterative refinement enhances the effectiveness of the testing process and increases overall API coverage.

The rest of this article is organized as follows. Section 2 provides a brief overview of the background and related work. Section 3 details the proposed approach for automatic generation of API sequences and parameter values. Section 4 presents and analyzes the experimental results. The conclusion remarks and future work are given in Section 5.

2. Background and Related Work

2.1. RESTler

RESTler [6] is a widely adopted stateful fuzzing tool for REST APIs to automatically test cloud services and uncover security and reliability issues. It analyzes OpenAPI specifications to identify producer–consumer dependencies by tracking how data flows from API responses to subsequent input parameters. The dependency information is used to generate valid and effective API call sequences. In testing, RESTler detects various types of errors, such as improper resource cleanup and failures due to invalid inputs. By iteratively extending API sequences, it explores deeper service states and uncovers more complex bugs.

Figure 1 illustrates the architecture of RESTler, which comprises two components: the RESTler Compiler and the RESTler Test Engine. The RESTler Compiler processes the OpenAPI specification to identify all request types, infer producer–consumer dependencies among APIs, and generate grammar files that guide the fuzzing process. The RESTler Test Engine uses these grammar files to construct test sequences, generate input values,

and execute the tests. It monitors API responses and incorporates feedback to refine and guide the generation of subsequent sequences. This iterative process continues until the predefined testing time limit is reached, at which point the final test results are produced.

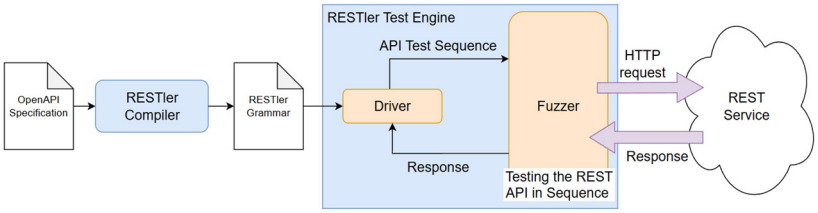


Figure 1. Architecture of RESTler.

2.2. Related Work

Recently, a growing body of research has focused on enhancing the automated testing of REST APIs. The following provides a brief review of existing work most relevant to our approach. Alonso et al. [7] utilized API parameter specifications and natural language processing techniques, combined with search-based and knowledge extraction methods, to automatically generate realistic test inputs. The authors defined syntactically and semantically valid inputs to ensure the correctness of the results. They then processed parameter names and descriptions to extract relevant search keywords, which were used to construct SPARQL queries to DBpedia to identify appropriate parameter predicates. Based on these predicates and pre-defined SPARQL templates, candidate input values were retrieved and applied to API calls. Experimental results demonstrated that retrieving input values from a knowledge base (DBpedia) significantly outperformed random input generation, yielding a greater number of valid values than other existing tools.

Wu et al. [8] proposed a fully automated and systematic approach for testing RESTful APIs based on combinatorial testing. Their method consists of a two-phase test case generation process. In the first phase, a sequence of constraints is established by identifying 'Create, Read, Update, Delete' (CRUD) relationships among API requests. In the second phase, four strategies are employed to generate input parameter values from the OpenAPI specification, and spaCy is utilized to infer dependencies among parameters. Once test cases are generated, combinatorial testing is applied to produce requests, and the process is iteratively repeated. Experimental results on 11 real-world RESTful services demonstrate that their approach outperforms existing tools in both effectiveness and efficiency.

Huang et al. [9] conducted a comprehensive study on the application of LLMs in fuzz testing techniques. Existing fuzzing methods are not fully automated, and as software vulnerabilities continue to evolve, the authors identified a growing trend toward LLM-based fuzzing. They overviewed approaches that integrate LLMs into software testing and fuzzing, supplemented by a statistical analysis of relevant literature and discussions on future potential. Techniques such as prompt engineering and seed mutation are highlighted as promising directions within this emerging field.

Karlsson et al. [10] proposed a simple and lightweight method for exploring the behavior of RESTful APIs. Their method automatically generates property-based tests from REST API specifications defined in OpenAPI documents. The results present how artifacts are constructed to serve both as test generators and as sources for result verification. Experimental results on both industrial and open-source services show that the method efficiently uncovers real bugs. Moreover, it enables the automatic detection of inconsistencies between API specifications and their implementations, offering deeper insights into the system under test. Since the tests are derived from OpenAPI documents, they automatically evolve alongside the REST API.

Viglianisi et al. [11] introduced RESTTESTGEN, a novel approach for automatically generating test cases from REST API documentation in Swagger format. For each API operation under test, the method generates corresponding input values and requests. Two distinct oracles are employed to detect errors during test execution. Experimental results demonstrate that this approach is effective in identifying real faults in real-world REST APIs. Laranjeiro et al. [12] presented a tool named bBOXRT, designed to conduct robustness testing on RESTful services using minimal information extracted from interface descriptions. The tool was evaluated on 52 REST services, encompassing 1351 operations across public, private, and internal APIs. The approach successfully uncovered various robustness issues, including deficiencies in services requiring high reliability as well as certain security vulnerabilities.

Martin-Lopez et al. [13,14] proposed RESTest, an automated black-box testing framework for RESTful APIs. Their approach identifies seven types of dependency relationships among request parameters and employs a constraint solver to address constraint satisfaction problems (CSPs) for test case generation. A novel testing oracle is applied to enhance error detection capabilities. Experimental results indicate that this method generates significantly more test cases than random testing and is capable of uncovering errors missed by random approaches. Notably, the framework successfully detected all known errors in the tested services.

In Ref. [15], we proposed a method to improve input generation for REST API testing by categorizing API parameters and generating fuzzed inputs based on their respective categories, rather than relying solely on random input generation. The parameter categories were derived through manual analysis of OpenAPI specifications collected from a broad set of popular RESTful services. During testing, each parameter is classified into a category using cosine similarity and metadata such as its name, type, format, and description. Inputs are then generated using category-specific rules, which include user-defined value lists and constraints specified in the OpenAPI documentation. Experimental results demonstrate that this approach enhances status code coverage and improves error detection compared to traditional random input generation.

3. Methodology

We reviewed the methodology, the API sequence generation method based on resource dependencies, and the parameter value generation and refinement using LLM.

3.1. Method Development

The developed method extends RESTler by incorporating resource dependencies among APIs and leveraging LLM-generated inputs to enhance API sequence construction and improve API coverage during testing. As illustrated in Figure 2, the process begins with parsing the OpenAPI specification of the service under test (SUT) using RESTler to generate a corresponding grammar file. This grammar is then provided to a configured RESTler engine to initiate the testing process.

In testing, API sequences are generated iteratively by incrementally increasing the sequence length. To construct these sequences, the proposed approach builds upon the producer–consumer dependencies inferred by RESTler—based on the relationship between API response object types and input parameters—and further incorporates dependencies among the resources manipulated by the APIs. By considering both types of dependencies, this approach avoids generating invalid or redundant API sequences, leading to more efficient and effective API testing. Once an API sequence is generated, the API endpoints, parameter names, and contextual information are embedded into a prompt template, which guides LLM to produce appropriate parameter values for each API call in the sequence. If

an error response is received during testing, the API endpoint, parameter context, and error message are used to construct a feedback prompt. This prompt helps refine the parameter inputs, which are then used in subsequent test iterations. This process continues until either the testing time limit is reached or all relevant API sequences have been executed. Finally, the test results are analyzed using RESTler.

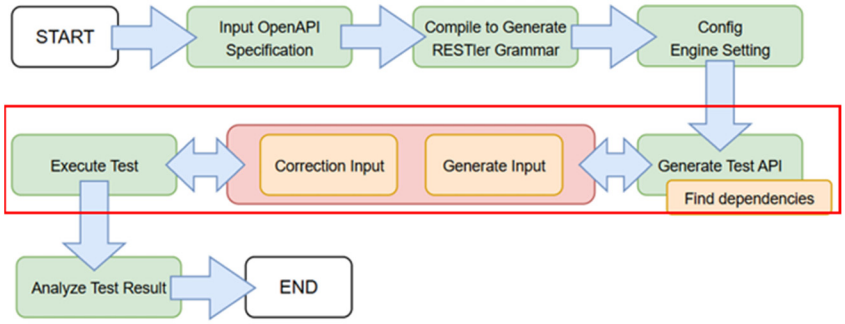


Figure 2. Overview of the proposed approach.

3.2. Generating API Sequence Using RESTler with Resource Dependency

The original RESTler identifies API dependencies solely from a dataflow perspective. For example, consider an API such as GET/project/{projectID}, where projectID is an object type used to retrieve project-specific information. If another API returns a projectID in its response, RESTler assumes a dependency exists and appends GET/project/{projectID} after that API in the sequence. This approach models dependencies as producer–consumer relationships, where an API that produces an object type in its response is considered a producer, and an API that consumes that object type as an input is considered a consumer.

However, this approach has several limitations. First, it treats any API that provides the required object type as a dependency of the API that consumes it—even when the two APIs operate on different resources—resulting in unnecessary or invalid API sequences. Second, it may incorrectly identify an API as dependent on itself. The analysis of APIs shows that object types present in an API's input parameters often also appear in its response. These issues can result in invalid or unnecessary API sequences during testing, ultimately reducing the efficiency and effectiveness of the testing process.

Since different APIs often operate on the same underlying resources, the order in which they are invoked may need to comply with specific business logic. For example, a resource generally must be created before it can be updated or deleted. By accounting for these resource-level relationships, the proposed approach facilitates the generation of more valid and meaningful API sequences, thereby improving both test coverage and overall testing effectiveness.

To identify resource-level dependencies among APIs, the OpenAPI specification is analyzed to construct an API resource tree, where each node represents a resource accessed by an API and is labeled with its corresponding endpoint (uniform resource identifier, URI). The parent–child relationships within the tree reflect the hierarchical organization of these resources. Typically, dependencies exist between parent and child nodes, as child resources are often nested within or logically linked to their parent resources. This hierarchical structure highlights the resource dependencies that must be considered during API execution and testing.

Figure 3 illustrates an example of an API resource tree. The list of APIs on the left-hand side of the figure includes APIs that operate at different levels of the resource hierarchy. For instance, the first API targets the projects resource, the second targets the repo resource under projects, and so on. By examining the URI path structures of these APIs, the API resource tree shown on the right-hand side of Figure 3 can be constructed automatically.

The tree's root node is at the top, with various resources branching out below—each node corresponding to a specific API. For example, the user node represents the user API nested under the projects resource, indicating a resource dependency between the projects and user APIs.

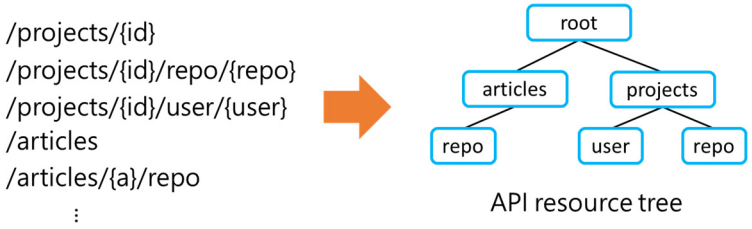


Figure 3. An example of an API resource tree.

3.3. Generating API Parameter Vaules Using LLM

RESTler generates parameter input values using a dictionary-based random approach. Although this method may work for testing individual APIs, it often fails when APIs have strict parameter constraints. If an API in a sequence fails to execute due to invalid inputs, any subsequent APIs that rely on its output cannot be tested, thereby reducing the overall effectiveness of the testing process. To address this problem, we introduced LLMs to generate valid parameter input values for APIs. Specifically, two prompt templates are designed to guide LLM toward performing the task effectively: the initial prompt, which is used to generate input values based on available API information, and the feedback prompt, which is used to refine those values when an API test fails. The details of each prompt design are described as follows.

3.3.1. Generating API Parameter Values Using Initial Prompts

The initial prompt is used to generate parameter inputs when testing an API for the first time. If the API call fails, the feedback prompt incorporates error messages and contextual information from previous attempts to help LLM refine the input values, increasing the likelihood of successful execution in subsequent tests.

Figure 4 illustrates an example of the initial prompt, which is divided into four key sections. The first section provides a clear and concise instruction to guide the generation of parameter input values. The second section presents a one-shot example that offers contextual information about a sample API, including its endpoint, parameter names, descriptions, and expected output. The third section provides a few-shot examples of parameters related to the target parameter, along with their corresponding API descriptions, to help LLM understand parameter dependencies when generating input values. The final section focuses on the target parameter itself, presenting its detailed information to support valid value generation.

To enable LLM to generate valid input values for a target parameter, two additional steps are performed before constructing the initial prompt. The first step is parameter name completion, which attempts to infer and reconstruct the full form of abbreviated parameter names using information from the parameter descriptions. In many OpenAPI specifications, shortened or abbreviated parameter names are used to reduce the overall API length. However, this practice can limit the amount of semantic information available, making it harder for LLM to accurately interpret the parameter's meaning. To mitigate this issue, we enriched abbreviated parameter names using information extracted from their associated descriptions. For example, if an API defines a parameter named "proj" with the description "A project ID", it can be reasonably inferred that "proj" refers to "project". When a word in the description matches or is semantically related to the parameter name,

we treat it as the full form and complete the parameter name accordingly to improve clarity and contextual understanding within the prompt.

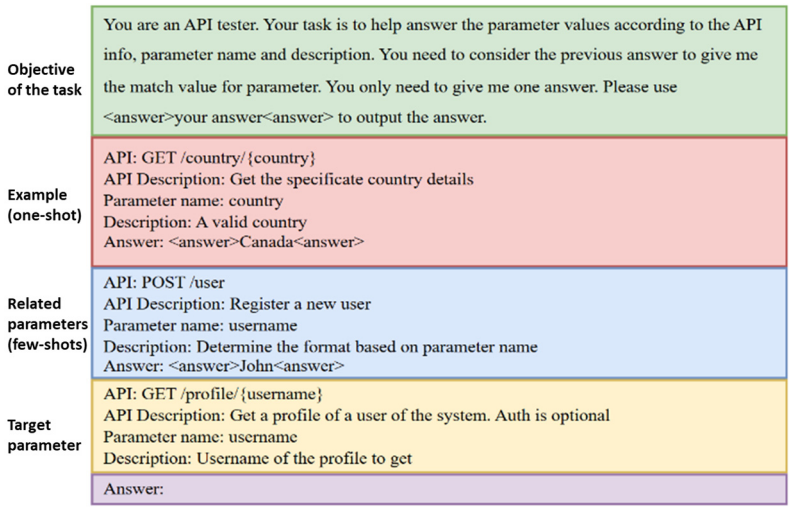


Figure 4. An example of an initial prompt used for generating API parameter values.

The second step involves identifying and extracting parameters related to the target parameter, along with their corresponding API descriptions. These related parameters provide additional context, increasing the likelihood that LLM generates valid and context-aware input values. In many cases, parameters exhibit dependencies. For example, the/event API may include both startDate and endDate parameters, where it is logically expected that startDate should not be later than endDate. Generating values for such parameters independently—without considering their interrelationship—can result in invalid inputs and cause the API call to fail. Therefore, recognizing and accounting for parameter dependencies is essential.

In this study, parameters defined within the same API are treated as interdependent and are included together when constructing prompts. Additionally, if a parameter's description contains the name of another parameter, the two are also considered related. For example, if one parameter is a project with the description "A project ID, . . ." and another is issues with the description "The project issues ID, . . .", the shared term "project" suggests a semantic relationship between the two parameters. This work leverages the occurrence of one parameter's name in another's description as an indicator of contextual dependency and includes the related parameters in the prompt accordingly.

3.3.2. Refining API Parameter Values Using Feedback Prompts

The parameter values generated by LLM can be invalid or fail to meet the API's input constraints due to insufficient context provided in the prompt. For example, consider a parameter named username with the description "A valid username". Using only this information, LLM generates a value like "name". However, during testing, API may return an error such as "The username must be at least 8 characters long and include at least one number". Because the original description lacked these specific requirements, LLM was unable to generate a valid input, leading to a failed API call.

In such cases, a feedback prompt is designed to help LLM refine previously generated input values. As shown in Figure 5, the feedback prompt follows a structure similar to the initial prompt, starting with a task instruction that clearly defines LLM's objective. It then provides a one-shot example that includes API details, parameter names, related parameters, previously generated values, error messages, and outputs to offer concrete guidance. Additionally, the prompt includes contextual information about the target

parameter—such as its description and any semantically related parameters that may influence its validity. By incorporating this information along with feedback from failed API calls (e.g., error messages), LLM is guided to revise or correct the input values. These refined values are then used in subsequent testing iterations, improving the success rate of API executions.

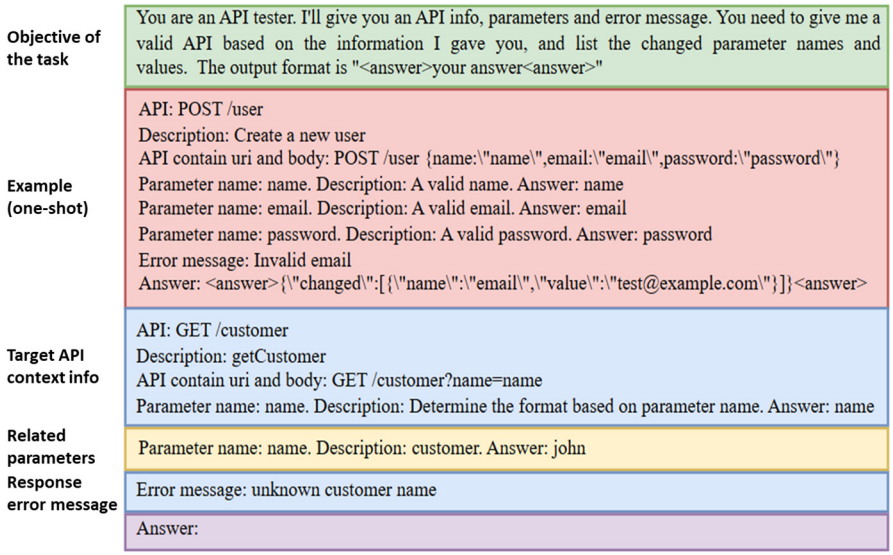


Figure 5. An example of a feedback prompt used for refining API parameter values.

4. Experiment

To evaluate the effectiveness of the proposed method, a series of experiments was conducted to address three key research questions. The first question investigates the extent to which the designed prompt template can guide a large language model (LLM) in generating valid API parameter values. The second question examines the influence of incorporating a feedback prompt on the outcomes of API testing. The third question explores whether the proposed method can achieve higher API coverage—specifically for 2XX (successful) and 5XX (server error) status codes—when compared to the performance of RESTler.

4.1. Experimental Environment

The experiments were conducted on a desktop PC running Windows 10, equipped with an Intel Core i7-9700 CPU @ 3.00 GHz, 48 gigabytes of random access memory, and an NVIDIA Quadro RTX 4000 graphics processing unit. The software and their corresponding versions used in the experiments are summarized in Table 1. In particular, the LLM used was the open-source Meta-Llama-3-8B model [16].

Table 1. The subject services selected from EvoMaster Benchmark (EBM).

Software	Version	Note
OS	Windows 11 Professional	23H2 ×64
Docker desktop	4.24.2	Used to set up the system under test (SUTs)
LLM	Meta-Llama-3-8B	-
llama.cpp	b3286	Used to run LLM
RESTler	9.2.0	-
Python	3.9.13	-
.NET	6.0.21	Used to build RESTler

For the selection of experimental subjects, we referred to the EvoMaster Benchmark (EMB) [17], from which only non-deprecated and containerizable targets were considered. The final set of SUTs (Table 2) includes those selected from EMB as well as GitLab, which is also used in RESTler.

Table 2. The subject services selected from EMB.

Services	Number of APIs
bibliothek	8
features-service	18
genome-nexus	23
market	13
proxyprint-kitchen	115
CatWatch	23
languagetool	5
disease.sh	31
nestjs-realworld-example-app (realworld)	19
GitLab	99

For the evaluation metric, we chose status code coverage as the primary indicator. Status code coverage refers to the HTTP status codes returned by the service under test in response to test requests. A 2XX code indicates a successful request, 3XX indicates redirection, 4XX indicates a client-side error, and 5XX indicates a server-side error. When an API request is sent and a response is received, the corresponding status code for that API is considered covered. For example, if a POST/user request returns a 200 status code, it means the 200 response for POST/user has been covered. If the same request later returns 200, it is not counted again, since that status code has already been covered for that API.

4.2. Experiment 1

To assess the effectiveness of the designed prompt template in guiding LLM to generate valid API parameter values, we selected 29 API parameters across 11 distinct categories, based on the parameter taxonomy identified from a total of 561 APIs in our previous study [15]. An LLM was then prompted to generate values for each of these parameters. The generated values were manually reviewed and considered valid if they satisfied the constraints of their respective parameters. The statistical results are summarized in Table 3. The results showed that all parameters except for String Date were successfully generated with values that conformed to their expected formats. Further analysis revealed that the failure of the String Date was due to an ambiguous description that specified two conflicting formats: the ISO 8601 [18] datetime format and a string date format represented as yyyy-mm-dd. This ambiguity made it difficult for LLM to determine the intended format, leading to an invalid output. These findings highlight the importance of clear and precise parameter descriptions in API documentation to support effective automated testing.

We compared the number of APIs returning HTTP status codes 2XX (successful responses) and 5XX (server error responses) for the subject services listed in Table 2, using the developed parameter value generation method versus RESTler's default dictionary-based approach for API sequences of length 1. The comparison results are presented in Table 4. The findings indicate that, on average, the parameter values generated by LLM result in a higher number of covered APIs for both 2XX and 5XX status codes.

Table 3. Statistical results of LLM-generated parameters.

Parameter Category	Rule	Parameter Count	Valid Generated Parameter Values
Area	List of coordinated universal time offsets	3	3
Uniform resource locator	Starting with http:// or https:// string	7	7
Language	ISO 639-1 [19]	1	1
Media type	Media type IANA list	1	1
Color	Hexadecimal color codes	2	2
Email	RFC5322	2	2
Path	File path string	1	1
Domain	1. Existing domain	1	1
IP	2. No existing domain	1	1
String datetime	IPv4	8	8
String date	ISO 8601	2	**1**

Table 4. Comparison of the number of covered APIs between the proposed method and RESTler's default dictionary-based approach for parameter value generation.

SUT	Approach	API Number (2XX)		API Number (5XX)	
		Count	% Increase	Count	% Increase
bibliothek	RESTler	1	0%	0	0%
	Proposed	1		0	
features-service	RESTler	14	0%	16	0%
	Proposed	14		16	
genome-nexus	RESTler	13	7.69%	8	12.5%
	Proposed	**14**		**9**	
market	RESTler	2	0%	7	0%
	Proposed	2		7	
proxyprint-kitchen	RESTler	40	0%	22	0%
	Proposed	40		22	
CatWatch	RESTler	8	0%	9	0%
	Proposed	8		9	
languagetool	RESTler	1	0%	1	0%
	Proposed	1		1	
disease.sh	RESTler	19	26.32%	0	0%
	Proposed	**24**		0	
realworld	RESTler	3	66.67%	4	50%
	Proposed	**5**		**6**	
GitLab	RESTler	25	12%	5	20%
	Proposed	**28**		**6**	
Average			11.3%		8.3%

The developed method generated API parameter values using LLM and demonstrated promising effectiveness. Using the designed prompt template, LLM generates values across various parameter categories, leading to improved API coverage. Compared with RESTler's default dictionary-based method, the LLM-based method increased the number of covered APIs returning 2XX status codes by an average of 11.3% and 5XX status codes by 8.3%.

4.3. Experiment 2

To evaluate the effectiveness of the feedback prompt, we configured the test to generate API sequences of up to length 3. This setup enabled the collection of API responses from earlier steps in the sequence (lengths 1 and 2), which were then integrated into the prompt

template to guide LLM in refining subsequent parameter values. We compared two scenarios: one without feedback and one with feedback embedded in the prompt. Table 5 presents the number of APIs covered and the percentage increase in 2XX and 5XX status code coverage for both settings. The results show that incorporating feedback prompts helps LLM revise parameter values more effectively, leading to higher API coverage for both 2XX and 5XX status codes.

Table 5. Comparison of API coverage with and without feedback.

SUT	Response Feedback	API Number (2XX)		API Number (5XX)	
		Count	% Increase	Count	% Increase
bibliothek	w/o feedback w/ feedback	1 1	0%	0 0	0%
features-service	w/o feedback w/ feedback	15 15	0%	16 16	0%
genome-nexus	w/o feedback w/ feedback	14 16	14%	9 9	0%
market	w/o feedback w/ feedback	2 2	0%	7 7	0%
proxyprint-kitchen	w/o feedback w/ feedback	40 40	0%	22 22	0%
CatWatch	w/o feedback w/ feedback	8 8	0%	9 9	0%
languagetool	w/o feedback w/ feedback	1 1	0%	1 1	0%
disease.sh	w/o feedback w/ feedback	24 24	0%	0 0	0%
realworld	w/o feedback w/ feedback	5 9	80%	6 8	33%
GitLab	w/o feedback w/ feedback	29 31	7%	6 7	17%
Average			10.1%		5.0%

Using feedback prompts to revise previously generated parameter values is effective. For API sequences of up to length 3, incorporating feedback into the prompts led to an average increase of 10.1% in the number of APIs returning 2XX status codes and 5.0% for 5XX status codes, compared to the scenario without feedback.

4.4. Experiment 3

In this experiment, each subject service listed in Table 2 was tested for six hours to compare the method that leverages API resource dependencies for sequence generation and uses the proposed prompt templates to guide LLM in generating and refining parameter values with RESTler. Table 6 presents the number of APIs covered and the percentage increase in 2XX and 5XX status code coverage for both approaches. The results demonstrate that the proposed method is more effective than RESTler, achieving a higher number of covered APIs for both 2XX and 5XX status codes. Compared with RESTler, the developed method showed an average increase of 9.0% in the number of APIs returning 2XX status codes and discovered 5.3% more APIs returning 5XX status codes. These findings demonstrate the effectiveness of the proposed method in improving API coverage for both successful (2XX) and server error (5XX) responses.

Table 6. Comparison of API coverage between our approach and RESTler.

SUT	Approach	API Number (2XX)		API Number (5XX)	
		Count	% Increase	Count	% Increase
bibliothek	RESTler	1	0%	0	0%
	Proposed	1		0	
features-service	RESTler	15	0%	16	0%
	Proposed	15		16	
genome-nexus	RESTler	13	23.08%	9	0%
	Proposed	**16**		9	
market	RESTler	2	0%	7	0%
	Proposed	2		7	
proxyprint-kitchen	RESTler	40	0%	22	0%
	Proposed	40		22	
CatWatch	RESTler	8	0%	9	0%
	Proposed	8		9	
languagetool	RESTler	1	0%	1	0%
	Proposed	1		1	
disease.sh	RESTler	19	26.32%	0	0%
	Proposed	**24**		0	
realworld	RESTler	7	28.57%	6	33.33%
	Proposed	**9**		**8**	
GitLab	RESTler	25	12%	5	20%
	Proposed	**28**		**6**	
Average			9.0%		5.3%

5. Conclusions and Future Work

We developed an automated approach to REST API testing that enhances both sequence construction and parameter input generation. By using an API resource tree, the method captures resource dependencies to build valid API sequences. It employs initial and feedback prompts to guide an LLM in generating and refining parameter inputs, leveraging techniques such as name completion, related parameter context, and error feedback. Experiments show that the proposed method improves API coverage and fault detection compared to existing approaches, demonstrating its effectiveness.

It is necessary to enhance API dependency inference and sequence generation by incorporating additional types of dependencies, such as authentication/authorization requirements and pagination logic. To further improve parameter input generation, external API documentation and domain-specific testing knowledge need to be integrated into the prompting process, thereby increasing the validity and effectiveness of the generated inputs.

Author Contributions: Conceptualization, C.-H.L.; methodology, C.-H.L.; software, K.-Y.L.; validation, C.-H.L., K.-Y.L. and S.-L.C.; formal analysis, C.-H.L. and K.-Y.L.; investigation, K.-Y.L.; resources, C.-H.L.; data curation, K.-Y.L.; writing—original draft preparation, C.-H.L., S.-L.C. and K.-Y.L.; writing—review and editing, S.-L.C.; visualization, S.-L.C.; supervision, S.-L.C.; project administration, S.-L.C.; funding acquisition, C.-H.L. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by the National Science and Technology Council (NSTC) of Taiwan, grant number NSTC 112-2221-E-027-049-MY2.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The original contributions presented in this study are included in the article. Further inquiries can be directed to the corresponding author.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Amazon AWS. Available online: https://aws.amazon.com/ (accessed on 8 May 2025).
2. Google Cloud Platform. Available online: https://cloud.google.com/ (accessed on 8 May 2025).
3. Microsoft Azure. Available online: https://azure.microsoft.com/ (accessed on 8 May 2025).
4. OpenAPI Specification. Available online: https://www.openapis.org (accessed on 15 June 2025).
5. REST. Available online: https://en.wikipedia.org/wiki/REST (accessed on 10 July 2025).
6. RESTler. Available online: https://github.com/microsoft/restler-fuzzer (accessed on 10 July 2025).
7. Alonso, J.C.; Martin-Lopez, A.; Segura, S.; Garcia, J.M.; Ruiz-Cortes, A. ARTE: Automated generation of realistic test inputs for web APIs. *IEEE Trans. Softw. Eng.* **2022**, *49*, 348–363. [CrossRef]
8. Wu, H.; Xu, L.; Niu, X.; Nie, C. Combinatorial testing of RESTful APIs. In Proceedings of the 44th International Conference on Software Engineering (ICSE), Pittsburgh, PA, USA, 25–27 May 2022; pp. 426–437.
9. Huang, L.; Zhao, P.; Chen, H.; Ma, L. Large language models based fuzzing techniques: A survey. *arXiv* **2024**, arXiv:2402.00350. [CrossRef]
10. Karlsson, S.; Čaušević, A.; Sundmark, D. QuickREST: Property-based test generation of OpenAPI-described RESTful APIs. In Proceedings of the IEEE 13th International Conference on Software Testing, Validation and Verification (ICST), Porto, Portugal, 23–27 March 2020; pp. 131–141.
11. Viglianisi, E.; Dallago, M.; Ceccato, M. Resttestgen: Automated black-box testing of RESTful APIs. In Proceedings of the IEEE 13th International Conference on Software Testing, Validation and Verification (ICST), Porto, Portugal, 23–27 March 2020; pp. 142–152.
12. Laranjeiro, N.; Agnelo, J.; Bernardino, J. A black box tool for robustness testing of REST services. *IEEE Access* **2021**, *9*, 24738–24754. [CrossRef]
13. Martin-Lopez, A.; Segura, S.; Ruiz-Cortés, A. RESTest: Black-box constraint-based testing of RESTful web APIs. In Proceedings of the 18th International Conference on Service-Oriented Computing (ICSOC 2020), Dubai, United Arab Emirates, 14–17 December 2020; pp. 459–475.
14. Martin-Lopez, A.; Segura, S.; Ruiz-Cortés, A. RESTest: Automated black-box testing of RESTful web APIs. In Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA), Virtual, Denmark, 11–17 July 2021; pp. 682–685.
15. Liu, C.H.; Chen, S.L.; Huang, H.K. Automated Test Input Generation for Testing Representational State Transfer (REST) Application Programming Interface (API) using Parameter Fuzzing. In Proceedings of the 6th IEEE International Conference on Knowledge Innovation and Invention 2023 (ICKII 2023), Sapporo, Japan, 11–13 August 2023; pp. 249–253.
16. Meta Llama 3. Available online: https://llama.meta.com/llama3/ (accessed on 19 July 2024).
17. EvoMaster Benchmark (EMB). Available online: https://github.com/WebFuzzing/EMB/ (accessed on 15 June 2025).
18. *ISO 8601*; Date and Time. International Organization for Standardization (ISO): Geneva, Switzerland, 2019.
19. *ISO 639-1*; Codes for the Representation of Names of Languages—Part 1: Alpha-2 Code. International Organization for Standardization (ISO): Geneva, Switzerland, 2002.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.

Copyright of Engineering Proceedings is the property of MDPI and its content may not be copied or emailed to multiple sites without the copyright holder's express written permission. Additionally, content may not be used with any artificial intelligence tools or machine learning technologies. However, users may print, download, or email articles for individual use.